

Beyond the Surrogate: Comparing Bayesian and Adaptive Coordinate-wise Hyperparameter Optimization

Dennis Diehl

March 24, 2026
Version: CleanThesis 0.4.1 - JGU patches



Johannes Gutenberg University Mainz
FB08
Institute of Computer Science
Data Mining

Bachelor thesis
submitted for the award of the degree
Bachelor of Science (B.Sc.)

Beyond the Surrogate: Comparing Bayesian and Adaptive Coordinate-wise Hyperparameter Optimization

Dennis Diehl

- 1. Reviewer* **Jane Doe**
Institute of Computer Science - Data Mining
Johannes Gutenberg University Mainz
- 2. Reviewer* **John Doe**
Department of Clean Thesis Style
Clean Thesis Style University
- Supervisors* Jane Doe and John Smith

March 24, 2026

Dennis Diehl

Beyond the Surrogate: Comparing Bayesian and Adaptive Coordinate-wise Hyperparameter Optimization

Bachelor thesis, March 24, 2026

Reviewers: Jane Doe and John Doe

Supervisors: Jane Doe and John Smith

Johannes Gutenberg University Mainz

Data Mining

Institute of Computer Science

FB08

Staudingerweg 9

55128 Mainz

Abstract

TODO: Write abstract (200 words). Content: motivation, research question, methods (MLP + RF, RS vs. BO, Breast Cancer), key findings (model quality + resource efficiency), conclusion.

Abstract (german)

Contents

1. Introduction	1
2. Related Work	3
2.1. The HPO Landscape	3
2.2. Exhaustive and Randomized Search	4
2.3. Surrogate Based Optimization	5
2.4. Coordinatewise and Adaptive Search Strategies	5
2.5. Comparative Studies and Research Gap	6
3. Preliminaries	9
3.1. Hyperparameter Formalization	9
3.2. The HPO Problem	9
3.3. Evaluation Metrics	10
3.4. Models	11
3.4.1. Multilayer Perceptron (MLP)	11
3.4.2. Random Forest	12
4. Methodology	13
4.1. Hyperparameter Search Spaces	13
4.1.1. MLP Search Space	13
4.1.2. Random Forest Search Space	14
4.2. Optimization Methods	14
4.2.1. Bayesian Optimization (TPE)	14
4.2.2. Adaptive Coordinate Random Search (ACRS)	16
4.3. Search Space Visualization	18
5. Experimental Setup and Datasets	19
5.1. Datasets	19
5.1.1. Breast Cancer Wisconsin	19
5.1.2. Pima Indians Diabetes	19
5.1.3. Bank Marketing	19
5.2. Experimental Setup	20
6. Results	21
6.1. Model Quality	21
6.2. Cross-Dataset Overview	22
6.3. Convergence Behavior	22
6.4. Resource Efficiency	23
6.4.1. Training Time	23
6.4.2. Energy Consumption	23

6.5. Quality–Efficiency Trade-off	23
6.6. Hyperparameter Sampling Distributions	24
7. Discussion	29
7.1. Comparison of Optimization Methods	29
7.2. Result Stability Across Seeds	29
7.3. Limitations	29
8. Conclusion	31
8.1. Summary of Findings	31
8.2. Practical Implications and Future Work	31
Bibliography	33
List of Figures	37
List of Tables	39
List of Listings	41
A. Appendix	43
A.1. Complete Results Tables	43
A.2. Best Hyperparameters per Configuration	43
A.3. Hardware and Software Environment	43
A.4. Reproducibility Checklist	43

Introduction

TODO: Write flat introductory chapter (no subsections), covering:

Opening paragraph: Importance of machine learning in practice; the critical role of hyperparameters in model performance; manual tuning as infeasible at scale → need for automated HPO methods. Cite [7], [3].

Research question: Does Bayesian Optimization (TPE) outperform Adaptive Coordinate Random Search (ACRS) — or vice versa — at an equal trial budget of 40 evaluations? Is the surrogate-model overhead of BO justified on small tabular datasets? Do findings generalise across both MLP and Random Forest, and across multiple datasets? Which method is more time- and energy-efficient? Reference [23].

Objectives: Empirical comparison of BO vs. ACRS on three benchmark datasets (Breast Cancer Wisconsin, Pima Indians Diabetes, Bank Marketing); 2 models (MLP, Random Forest) × 2 optimisers × 3 seeds × 3 datasets = 36 runs, 40 trials each (1440 total evaluations); metrics: AUROC (primary), Accuracy, training time, energy (kWh via CodeCarbon). Reproducibility: seeds [42, 123, 456].

Thesis outline: Chapter 2 – Related Work surveys existing HPO methods and positions this work relative to prior comparisons; Chapter 3 – Preliminaries formalises HPO, introduces evaluation metrics, and describes the MLP and Random Forest models; Chapter 4 – Methodology details the two optimisation strategies (Bayesian Optimisation, ACRS) and the hyperparameter search spaces; Chapter 5 – Experimental Setup and Datasets presents the three benchmark datasets and the full experimental design (2 models × 2 optimisers × 3 seeds × 3 datasets); Chapter 6 – Results reports Val-AUROC, accuracy, training time, and energy consumption across all conditions; Chapter 7 – Discussion interprets the results and discusses limitations; Chapter 8 – Conclusion summarises findings and outlines future work.

Related Work

This chapter surveys the landscape of hyperparameter optimization methods, progressing from classical exhaustive and randomized approaches to surrogate based strategies such as Bayesian Optimization. It then introduces coordinatewise adaptive search as a distinct methodological family before examining existing comparative studies and identifying the research gap that the present work addresses.

2.1 The HPO Landscape

Hyperparameter optimization (HPO) aims to find a configuration of hyperparameters that maximizes the generalization performance of a machine learning model. A broad taxonomy of HPO methods has emerged over the past two decades, which can be organized along two principal axes: whether the method maintains a surrogate model of the objective function, and how it selects candidate configurations [7, 25].

Surrogate free methods evaluate configurations without modeling the relationship between hyperparameters and performance. Grid Search exhaustively evaluates a predefined lattice of configurations, while Random Search samples configurations independently from specified distributions. Neither approach leverages information from previously evaluated configurations to guide future sampling. In contrast, surrogate based methods construct a probabilistic model that approximates the mapping from hyperparameters to performance and use it to propose promising candidates. Bayesian Optimization (BO) is the most prominent representative of this family: it fits a surrogate—such as a Gaussian Process [20], a Tree-structured Parzen Estimator (TPE) [2], or a Random Forest (Sequential Model-based Algorithm Configuration, SMAC) [9]—and uses an acquisition function to balance exploration and exploitation when proposing the next configuration [18].

Beyond these core families, bandit based methods such as Hyperband [11] allocate budgets adaptively by terminating poorly performing configurations early, while evolutionary approaches and Population-Based Training mutate and recombine configurations across generations. The present work focuses on the comparison between BO (specifically TPE) and a coordinatewise adaptive search strategy (ACRS) that combines random sampling with adaptive parameter prioritization and range shrinking; evolutionary and bandit based methods are therefore outside the scope of this thesis but are mentioned here

for completeness. The following sections discuss each relevant methodological family in detail.

2.2 Exhaustive and Randomized Search

Grid Search is the classical exhaustive approach to HPO. The practitioner specifies a finite set of candidate values for each hyperparameter, and the algorithm evaluates every point in the Cartesian product of these sets. Because the evaluation order is fully deterministic, Grid Search is straightforward to implement and guarantees reproducibility: given identical grids and identical data splits, the same optimum is always returned [7]. The principal limitation is the curse of dimensionality. For a grid of n values per hyperparameter over d hyperparameters, the total number of evaluations is n^d , which grows exponentially and becomes impractical for $d \geq 5$ [25]. Moreover, many evaluations are wasted when only a small subset of hyperparameters materially affects performance, because the grid allocates equal resolution to every dimension regardless of its importance. Despite these shortcomings, Grid Search remains a widely used conceptual baseline in the HPO literature. Notably, the coordinatewise refinement principle—evaluating candidates along one dimension at a time while holding the others fixed—originates in Grid Search and also informs the ACRS strategy evaluated in this thesis.

Bergstra and Bengio [3] demonstrated that Random Search can substantially outperform Grid Search when the objective function has low effective dimensionality—that is, when only a few hyperparameters strongly influence performance. The key insight is geometric: in a grid, every sample shares coordinate values with many other samples along each axis, so the number of distinct values explored per important dimension equals only the grid resolution n . Random Search, by contrast, draws each configuration independently, so k trials explore up to k unique values along every dimension simultaneously. Random Search is trivially parallelizable because trials are independent, requires no memory of past evaluations, and offers flexible budget allocation: adding or removing trials does not invalidate previously collected results [2]. However, precisely because Random Search does not learn from prior evaluations, it cannot concentrate sampling in promising regions of the search space. This limitation motivates surrogate based methods such as Bayesian Optimization.

Both Grid Search and Random Search serve as important conceptual foundations for this work. Grid Search contributes the coordinatewise evaluation principle, while Random Search provides the sampling strategy used within each coordinate. Neither method is employed as a standalone comparison method in the experiments; instead, their principles are combined in the ACRS strategy described in Section 2.4.

2.3 Surrogate Based Optimization

Bayesian Optimization (BO) addresses the sample inefficiency of surrogate free methods by maintaining a probabilistic surrogate model of the objective function and using an acquisition function to decide which configuration to evaluate next. It has emerged as one of the most effective approaches for the efficient global optimization of expensive black-box functions [14]. The surrogate–acquisition loop enables BO to concentrate evaluations in promising regions while preserving exploration of uncertain areas of the search space [18].

The canonical instantiation is GP based BO, popularized for hyperparameter tuning by Snoek et al. [20], which uses a Gaussian Process as the surrogate and Expected Improvement (EI) as the acquisition function. GP based BO provides calibrated uncertainty estimates but scales cubically with the number of observations, which can become a bottleneck for large evaluation budgets.

The Tree-structured Parzen Estimator (TPE), introduced by Bergstra et al. [2], addresses this scaling limitation with a fundamentally different surrogate formulation: instead of modeling $p(y | \lambda)$ directly, TPE models two densities $\ell(\lambda)$ and $g(\lambda)$ —one over configurations whose observed performance exceeds a quantile threshold (*good* observations) and one over the rest—and selects candidates that maximize the ratio $\ell(\lambda)/g(\lambda)$, which can be shown to serve as a proxy for Expected Improvement. Because TPE fits independent univariate densities per hyperparameter, it handles conditional and categorical parameters naturally and scales more favorably than GPs as the number of observations grows. The detailed mechanism of TPE, including its density estimation procedure and candidate selection, is presented in Chapter 4 (Algorithm 1). A third BO variant, SMAC [9], replaces the GP with a Random Forest surrogate, offering native support for categorical parameters and robustness in configuration spaces with many dimensions.

This thesis employs TPE as the BO surrogate for three reasons: (1) it natively supports the mixed search spaces used in the experiments, which combine continuous, integer, and categorical hyperparameters; (2) it scales better than GP based BO under the moderate evaluation budget of 40 trials per run; and (3) it is efficiently implemented in Optuna [1], which provides a define-by-run search space specification and is orchestrated via Ray Tune [12] for distributed execution.

2.4 Coordinatewise and Adaptive Search Strategies

Coordinate descent is a classical optimization principle in which one variable is optimized at a time while all others remain fixed [24]. Applied to HPO, this translates to evaluating candidate values along a single hyperparameter

dimension, selecting the best, fixing it, and moving to the next dimension. This coordinatewise evaluation strategy is implicit in Grid Search, where the Cartesian product can be viewed as a nested loop over individual dimensions. However, a full coordinatewise grid still scales multiplicatively; the key advantage emerges when the Cartesian product is abandoned in favor of sequential per-dimension sweeps, reducing the total number of evaluations from n^d to $n \cdot d$.

Combining this coordinatewise principle with random sampling—drawing candidates from a distribution rather than a fixed grid—preserves the geometric advantage identified by Bergstra and Bengio [3]: each trial contributes a unique value along the dimension under consideration. This yields a search procedure that is both more sample efficient than full Grid Search and more focused than pure Random Search.

Adaptive extensions introduce two further mechanisms. First, the order in which dimensions are swept can be determined by observed importance rather than an arbitrary sequence: dimensions whose variation produced the largest performance change in prior rounds are prioritized, concentrating the remaining budget on the most influential hyperparameters. Second, search ranges can be shrunk around the current best value after each sweep, progressively refining the resolution in promising regions.

The Adaptive Coordinate Random Search (ACRS) strategy evaluated in this thesis combines these ideas into a surrogate free two-phase procedure. An initial Random Search phase explores the full search space to establish a performance baseline and provides observations that inform hyperparameter prioritization. A subsequent coordinatewise phase then refines the best configuration by sweeping one dimension at a time, ordered by observed importance and with adaptively narrowed ranges. The formal definition and pseudocode of ACRS are presented in Chapter 4 (Algorithm ??).

2.5 Comparative Studies and Research Gap

A number of studies have compared HPO methods empirically. Yang and Shami [25] survey both theory and practice of HPO algorithms and find that the relative ranking of methods varies across tasks and budgets, echoing the No Free Lunch theorem [23]. Feurer and Hutter [7] provide a comprehensive review of HPO within the broader AutoML landscape, noting that surrogate based methods generally outperform surrogate free baselines given sufficient budget, but that the margin depends heavily on the dimensionality and structure of the search space. Li et al. [11] propose Hyperband, a bandit based approach that complements BO by terminating unpromising configurations early, and demonstrate competitive performance with significantly fewer resources on several benchmarks.

Despite this body of work, several gaps remain:

1. Few studies systematically compare Bayesian Optimization against lightweight, surrogate free coordinatewise strategies that adaptively prioritize hyperparameters and shrink their search ranges.
2. The energy cost of the optimization process itself is rarely measured; tools such as CodeCarbon [17] have made carbon tracking feasible, but most HPO benchmarks still omit resource consumption metrics.
3. Small tabular datasets are underrepresented in the HPO literature, which tends to focus on large scale deep learning benchmarks.
4. Seed variability across repeated runs is seldom reported explicitly, making it difficult to distinguish genuine algorithmic differences from random fluctuations.
5. No existing benchmark directly compares BO and an adaptive coordinatewise random search strategy on multiple small tabular classification datasets under an equal trial budget.

The present work addresses these gaps by benchmarking TPE based Bayesian Optimization against Adaptive Coordinate Random Search across three tabular datasets, using three random seeds per configuration and tracking both predictive performance and energy consumption. The formal definitions of the optimization algorithms, search spaces, and evaluation protocol are presented in the following chapters.

Preliminaries

This chapter introduces the formal foundations underlying the experiments presented in this thesis. It defines the hyperparameter optimization problem, describes the evaluation metrics used to assess both predictive quality and computational cost, and presents the two model architectures—Multilayer Perceptron and Random Forest—whose hyperparameters are optimized. The concrete search spaces and optimization algorithms are deferred to Chapter 4.

3.1 Hyperparameter Formalization

Consider a learning algorithm A that, given a training dataset D_{train} and a hyperparameter configuration λ , produces a trained model $A(D_{\text{train}}, \lambda)$ with learned parameters θ (e.g. neural network weights or decision tree split points). The parameters θ are optimized internally by the algorithm—for instance via gradient descent in neural networks—whereas the hyperparameters λ must be specified before training begins and govern structural or procedural choices such as the learning rate, the number of hidden layers, or the splitting criterion [7].

Each hyperparameter λ_i takes values from a domain Λ_i that may be continuous ($\Lambda_i \subseteq \mathbb{R}$), integer-valued ($\Lambda_i \subseteq \mathbb{Z}$), or categorical ($\Lambda_i = \{v_1, \dots, v_k\}$). The full configuration space is the Cartesian product

$$\Lambda = \Lambda_1 \times \Lambda_2 \times \dots \times \Lambda_d, \quad (3.1)$$

and a concrete configuration is a vector $\lambda = (\lambda_1, \dots, \lambda_d) \in \Lambda$. In the present work, the MLP search space comprises $d = 8$ hyperparameters and the Random Forest search space $d = 7$ hyperparameters, mixing all three domain types (see Chapter 4).

3.2 The HPO Problem

The goal of hyperparameter optimization is to find the configuration that maximizes a validation metric $\mathcal{V}$ evaluated on a held-out validation set D_{val} :

$$\lambda^* = \arg \max_{\lambda \in \Lambda} \mathcal{V}(A(D_{\text{train}}, \lambda), D_{\text{val}}). \quad (3.2)$$

In this thesis, $\mathcal{V}$ is the validation AUROC (defined in Section 3.3).

Three properties make this optimization problem challenging. First, the objective is a *black box*: no gradient of $\mathcal{V}$ with respect to λ is available, because $\mathcal{V}$ depends on the entire training procedure. Second, each evaluation is *expensive*, as it requires a complete training run of the underlying model. Third, the No Free Lunch theorem [23] implies that no single optimization strategy dominates all others across all possible objective landscapes. Chapter 2 surveys the main families of methods that address this problem.

3.3 Evaluation Metrics

This study evaluates hyperparameter configurations along four dimensions: two predictive-quality metrics (AUROC and Accuracy) and two resource-consumption metrics (Training Time and Energy Consumption).

AUROC. The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate against the False Positive Rate at varying classification thresholds, where

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (3.3)$$

The Area Under the ROC Curve is defined as

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x)) dx. \quad (3.4)$$

AUROC is threshold-independent and robust under moderate class imbalance, making it preferable to accuracy for datasets with unequal class distributions [6]. It serves as the primary optimization target in all experiments.

Accuracy. Classification accuracy is defined as the fraction of correctly classified instances:

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (3.5)$$

Accuracy is reported as a secondary metric alongside AUROC for comparability with studies that use it as their primary measure.

Training Time. Wall-clock training time captures the duration of a single trial, i.e. one complete training run with a fixed hyperparameter configuration. It provides a hardware-dependent but practically relevant measure of computational cost.

Energy Consumption. Energy consumption is recorded in kilowatt-hours using CodeCarbon [17]. Conceptually, the energy consumed during a trial of duration T is

$$E = \int_0^T P(t) dt, \quad (3.6)$$

where $P(t)$ denotes the instantaneous power draw. CodeCarbon approximates this integral by periodically sampling CPU and, if available, GPU power counters. On CPU-only hardware—as used in this thesis—the precision of these estimates is limited, so energy figures should be interpreted as approximate indicators rather than exact measurements.

3.4 Models

This study employs two model families that represent fundamentally different learning paradigms: a gradient-based neural network (Multilayer Perceptron) and an ensemble of decision trees (Random Forest). Their distinct training mechanisms—iterative gradient descent versus parallel bootstrap aggregation—lead to qualitatively different hyperparameter landscapes and make the comparison of optimization strategies more informative.

3.4.1 Multilayer Perceptron (MLP)

A Multilayer Perceptron is a feedforward neural network consisting of an input layer, L hidden layers, and an output layer [8]. Given an input $h^{(0)} = x \in \mathbb{R}^d$, each hidden layer l computes

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right), \quad l = 1, \dots, L, \quad (3.7)$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, and σ is a nonlinear activation function. In this work, $\sigma \in \{\text{ReLU}, \tanh\}$.

To reduce overfitting, Dropout [21] is applied after each activation:

$$\tilde{h}^{(l)} = \text{Dropout}(h^{(l)}, p), \quad (3.8)$$

where each unit is independently set to zero with probability p during training and all units are retained (with rescaling) at inference time.

The output layer produces logits for the two classes:

$$\hat{y} = W^{(L+1)}\tilde{h}^{(L)} + b^{(L+1)} \in \mathbb{R}^2. \quad (3.9)$$

The network is trained by minimizing the cross-entropy loss

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\text{softmax}(\hat{y})_c), \quad (3.10)$$

where y_c is the one-hot encoded ground truth label. Three optimizers are considered: Adam [10], which adapts per-parameter learning rates using estimates of the first and second moments of the gradient; SGD with momentum 0.9; and AdamW, a variant of Adam with decoupled weight decay. All optimizers support an explicit weight decay term for L_2 regularization.

Training employs *early stopping* [16]: the validation loss is monitored after each epoch, and training is halted if no improvement is observed for 10 consecutive epochs (patience). The model state corresponding to the lowest validation loss is saved and restored at the end of training.

3.4.2 Random Forest

A Random Forest [5] is an ensemble of B decision trees, each trained on a bootstrap sample of the training data—a procedure known as *bagging* [4]. The ensemble prediction for an input x is obtained by majority vote:

$$\hat{y}(x) = \text{mode}\{T_b(x)\}_{b=1}^B. \quad (3.11)$$

Each tree is grown by recursively partitioning the feature space. At every node, only a random subset of features is considered as split candidates, which decorrelates the individual trees and reduces variance. The feature subset size is controlled by a dedicated parameter; typical choices are $\sqrt{d}$ and $\log_2 d$, where d is the number of input features.

Splits are evaluated using an impurity criterion. The two criteria available in this study are Gini impurity,

$$I_G(p) = \sum_{c=1}^C p_c(1 - p_c), \quad (3.12)$$

and entropy,

$$I_E(p) = - \sum_{c=1}^C p_c \log_2 p_c, \quad (3.13)$$

where p_c is the proportion of class c samples at a given node.

Unlike the MLP, the Random Forest is a non-iterative method: trees are grown to completion (subject to depth and leaf-size constraints) without an epoch-based training loop. Consequently, no early stopping is needed, and training times are substantially shorter than for the MLP.

Methodology

TODO: Write introductory paragraph giving an overview of the experimental design: 2 models $\times$ 2 optimisers (BO, ACRS) $\times$ 3 seeds $\times$ 3 datasets = 36 runs, 40 trials each (1440 total evaluations). Refer forward to Chapter 5 for the datasets and full experimental setup.

4.1 Hyperparameter Search Spaces

TODO: Explain how the search spaces were derived and justify the chosen ranges (e.g. log-uniform for learning rate due to multiple orders of magnitude; log-uniform for weight decay for the same reason).

4.1.1 MLP Search Space

Tab. 4.1.: Hyperparameter search space of the MLP.

Hyperparameter	Distribution	Range / Set	Scale
learning_rate	Log-Uniform	$[10^{-5}, 10^{-1}]$	logarithmic
batch_size	Categorical	{16, 32, 64, 128}	–
hidden_dim	Uniform Integer	[32, 256]	linear
dropout	Uniform	[0.0, 0.5]	linear
num_layers	Categorical	{1, 2, 3}	–
optimizer_name	Categorical	{adam, sgd, adamw}	–
weight_decay	Log-Uniform	$[10^{-6}, 10^{-2}]$	logarithmic
activation	Categorical	{relu, tanh}	–

4.1.2 Random Forest Search Space

Tab. 4.2.: Hyperparameter search space of the Random Forest.

Hyperparameter	Distribution	Range / Set	Scale
n_estimators	Uniform Integer	[50, 500]	linear
max_depth	Uniform Integer	[3, 30]	linear
min_samples_split	Uniform Integer	[2, 20]	linear
max_features	Categorical	{sqrt, log2}	–
min_samples_leaf	Uniform Integer	[1, 10]	linear
criterion	Categorical	{gini, entropy}	–
max_samples	Uniform	[0.5, 1.0]	linear

4.2 Optimization Methods

Both algorithms share the following notation, which is used throughout this section and in the pseudocode listings below:

- Λ — the hyperparameter search space, i.e. the set of all admissible configurations as defined in Tables 4.1 and 4.2.
- $f: \Lambda \rightarrow \mathbb{R}$ — the objective function that maps a configuration $\lambda \in \Lambda$ to its validation performance (Val-AUROC).
- T — the evaluation budget, i.e. the total number of trials ($T = 40$ in our setup).
- λ — a single hyperparameter configuration (a vector drawn from Λ).

4.2.1 Bayesian Optimization (TPE)

Bayesian Optimization (BO) uses a probabilistic surrogate model to guide the search toward promising regions of the hyperparameter space. Rather than evaluating configurations independently, BO maintains a model of the relationship between hyperparameters and performance and uses this model to decide which configuration to evaluate next. The variant used in this work employs the Tree-structured Parzen Estimator (TPE) [2] as its surrogate model, integrated via the Optuna framework [1]. The rationale for choosing TPE over Gaussian Process-based BO is discussed in Section 2.3.

The goal of BO is to find the configuration $\lambda^* \in \Lambda$ that maximises $f(\lambda)$ while minimising the number of expensive evaluations, by exploiting the surrogate model to concentrate samples near promising regions.

In addition to the shared notation, the TPE procedure uses the following variables:

- n_{warmup} — the number of initial random trials used to initialise the density estimators (default: 10).
- $\mathcal{D}$ — the set of observed (configuration, performance) pairs collected so far.
- $\ell(\lambda), g(\lambda)$ — the two kernel density estimators fitted to the *good* and *bad* observations respectively, separated at a quantile threshold.

Algorithm 1 summarises the procedure.

Algorithm 1 Bayesian Optimization via TPE

Require: Search space Λ , objective f , budget T , warm-up count n_{warmup}

```

1:  $\mathcal{D} \leftarrow \emptyset$ 
2: for  $t = 1$  to  $T$  do
3:   if  $t \leq n_{\text{warmup}}$  then
4:      $\lambda_t \leftarrow \text{Uniform}(\Lambda)$ 
5:   else
6:     Fit  $\ell(\lambda)$  and  $g(\lambda)$  on  $\mathcal{D}$ 
7:      $\lambda_t \leftarrow \arg \max_{\lambda} \ell(\lambda)/g(\lambda)$ 
8:   end if
9:    $y_t \leftarrow f(\lambda_t)$ 
10:   $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\lambda_t, y_t)\}$ 
11: end for
12: return  $\lambda^* = \arg \max_{(\lambda, y) \in \mathcal{D}} y$ 

```

Rather than modelling the objective $p(y \mid \lambda)$ directly, TPE maintains two separate kernel density estimators: $\ell(\lambda)$, fitted to configurations whose observed performance exceeds a quantile threshold (*good* observations), and $g(\lambda)$, fitted to the remaining *bad* observations. New candidates are drawn by maximising the ratio $\ell(\lambda)/g(\lambda)$, which can be shown to serve as a proxy for the Expected Improvement acquisition function [20].

Line 1 initialises an empty observation set $\mathcal{D}$. The main loop (lines 2–10) iterates over the budget of T trials. During the warm-up phase ($t \leq n_{\text{warmup}}$, lines 3–4) configurations are drawn uniformly at random from Λ ; this ensures that the density estimators have a sufficient data basis before they are used to guide the search. Once enough observations have been collected (lines 5–7), $\ell(\lambda)$ and $g(\lambda)$ are fitted on $\mathcal{D}$ and the next candidate is chosen by maximising the density ratio $\ell(\lambda)/g(\lambda)$ (TPE-guided sampling). Each candidate λ_t is then evaluated via the objective function (line 8), and the resulting pair (λ_t, y_t) is added to $\mathcal{D}$ (line 9). After all trials have been exhausted, the algorithm returns the configuration with the highest observed performance (line 11).

In our setup, Val-AUROC is maximised and each run evaluates $T = 40$ configurations. Trials are executed sequentially to ensure that every new suggestion can condition on all previous observations.

4.2.2 Adaptive Coordinate Random Search (ACRS)

Adaptive Coordinate Random Search (ACRS) is a coordinate-wise optimisation strategy that combines three core ideas. First, it optimises one hyperparameter at a time while holding all others fixed — a principle known as *coordinate-wise optimisation*, analogous to coordinate descent in continuous optimisation. Second, it maintains *adaptive weights* that track how much improvement each hyperparameter has contributed; parameters that yielded larger gains in previous rounds are prioritised through a weighted random permutation of the processing order. Third, it applies a *shrinking search range*: after each coordinate step the admissible interval for that hyperparameter is contracted around the current best value by a factor α , so that the search becomes progressively more refined.

The goal of ACRS is to find the configuration $\lambda^* \in \Lambda$ that maximises $f(\lambda)$ by iteratively refining individual hyperparameters, concentrating the search budget on the dimensions that contribute most to performance improvement.

In addition to the shared notation, ACRS uses the following variables:

- n — the dimensionality of the search space (number of hyperparameters, i.e. $n = |\Lambda|$).
- L — the number of random samples drawn per hyperparameter in each coordinate step (inner loop).
- R — the number of complete rounds over all hyperparameters (outer loop).
- $\alpha \in (0, 1)$ — the shrinking factor that controls how quickly the search range contracts. A small value (e.g. $\alpha = 0.3$) causes rapid contraction; a large value (e.g. $\alpha = 0.8$) preserves a wider range.
- w_j — the adaptive weight for hyperparameter j , reflecting its relative importance based on observed improvements.
- $\mathcal{R}_j$ — the current search range for hyperparameter j , initialised to the full domain and shrunk after each coordinate step.
- Δ_j — the improvement in f achieved during the coordinate step for hyperparameter j .

Algorithm 2 details the procedure.

Algorithm 2 Adaptive Coordinate Random Search (ACRS)

Require: Search space Λ with n hyperparameters, objective f , samples per parameter L , rounds R , shrinking factor $\alpha \in (0, 1)$

```
1:  $\lambda^* \leftarrow \text{Random}(\Lambda)$ ;  $y^* \leftarrow f(\lambda^*)$ 
2:  $w_j \leftarrow 1/n$  for all  $j = 1, \dots, n$ 
3:  $\mathcal{R}_j \leftarrow \Lambda_j$  for all  $j = 1, \dots, n$ 
4: for  $r = 1$  to  $R$  do
5:    $\Delta_j \leftarrow 0$  for all  $j = 1, \dots, n$ 
6:    $\pi \leftarrow \text{WeightedPermutation}(\{1, \dots, n\}, \mathbf{w})$ 
7:   for  $j \in \pi$  do
8:      $y_{\text{before}} \leftarrow y^*$ 
9:     for  $l = 1$  to  $L$  do
10:       $\lambda' \leftarrow \lambda^*$  with  $\lambda'_j \leftarrow \text{Sample}(\mathcal{R}_j)$ 
11:       $y' \leftarrow f(\lambda')$ 
12:      if  $y' > y^*$  then
13:         $y^* \leftarrow y'$ ;  $\lambda^* \leftarrow \lambda'$ 
14:      end if
15:    end for
16:     $\Delta_j \leftarrow y^* - y_{\text{before}}$ 
17:     $\mathcal{R}_j \leftarrow \text{Shrink}(\mathcal{R}_j, \lambda_j^*, \alpha)$ 
18:  end for
19:   $\text{total} \leftarrow \sum_{j=1}^n \Delta_j$ 
20:  if  $\text{total} > 0$  then
21:     $w_j \leftarrow \Delta_j / \text{total}$  for all  $j$ 
22:  else
23:     $w_j \leftarrow 1/n$  for all  $j$ 
24:  end if
25: end for
26: return  $\lambda^*, y^*$ 
```

Lines 1–3 initialise the algorithm: a random configuration is drawn from the full search space and evaluated, the adaptive weights are set uniformly to $1/n$, and the search ranges are initialised to the full domain of each hyperparameter.

The outer loop (lines 4–21) iterates over R rounds. At the start of each round, the per-parameter improvements Δ_j are reset to zero (line 5), and the processing order π is determined by a weighted random permutation based on the current weights $\mathbf{w}$ (line 6). Parameters with higher weights are more likely to appear early in the permutation, ensuring that the algorithm devotes its budget preferentially to the most impactful dimensions.

The inner loop (lines 7–16) processes each hyperparameter j in the order given by π . For each hyperparameter, L candidate values are sampled uniformly from the current range $\mathcal{R}_j$ (line 10), while all other hyperparameters remain fixed at their best-known values. Whenever a candidate improves

upon the incumbent, λ^* and y^* are updated (lines 12–13). After all L samples, the improvement Δ_j is recorded (line 15) and the search range $\mathcal{R}_j$ is contracted around the current best value λ_j^* using the shrinking factor α (line 16). $\text{Sample}(\mathcal{R}_j)$ draws a value uniformly at random from the current range $\mathcal{R}_j$; for continuous hyperparameters this is a uniform draw over the interval, for categorical hyperparameters it is a uniform draw over the remaining candidates. $\text{Shrink}(\mathcal{R}_j, \lambda_j^*, \alpha)$ contracts the range around the current best value: for a continuous range $[a, b]$ the new range is $[\lambda_j^* - \alpha(\lambda_j^* - a), \lambda_j^* + \alpha(b - \lambda_j^*)]$, clipped to the original bounds; for categorical hyperparameters the set is left unchanged.

After completing a full pass over all hyperparameters, the total improvement total is computed as the sum of all per-parameter improvements (line 17). The weights are then updated (lines 18–21): if any improvement was observed, each weight is set proportionally to its contribution; otherwise, the weights are reset to uniform. This adaptive mechanism ensures that hyperparameters which consistently drive performance gains receive increasing attention in subsequent rounds.

The total number of objective evaluations is $1 + R \cdot n \cdot L$ (one initial evaluation plus L samples per parameter per round). The key distinction from Bayesian Optimisation is the absence of a probabilistic surrogate model: ACRS relies on coordinate-wise random sampling with adaptive prioritisation and range shrinking, trading model-based guidance for a lightweight, model-free search strategy.

4.3 Search Space Visualization

TODO: Explain the conceptual illustration in Figure ??: the 2D projection onto learning rate (log-scale) and hidden dimension axes. Discuss how Bayesian Optimization (left panel) begins with a uniform warm-up phase and then concentrates samples near promising regions via the TPE density ratio $\ell(\lambda)/g(\lambda)$, while ACRS (right panel) optimises one coordinate at a time with adaptive sampling and shrinking ranges.

Experimental Setup and Datasets

TODO: Write a short introductory paragraph: this chapter describes the three datasets used in the study and the full experimental design under which the methods of Chapter 4 are evaluated.

5.1 Datasets

TODO: Briefly motivate the choice of three binary classification datasets with different sizes, feature counts, and class distributions. Explain that AUROC is the primary metric precisely because all three datasets exhibit at least mild class imbalance.

5.1.1 Breast Cancer Wisconsin

TODO: Describe the Breast Cancer Wisconsin (Diagnostic) dataset: 569 samples, 30 numerical features, 2 classes (357 benign, 212 malignant). Source: UCI ML Repository. Split: 60 % train / 20 % val / 20 % test (stratified). Scaling: standard scaling (fit on train, transform on val/test). Reference [22].

5.1.2 Pima Indians Diabetes

TODO: Describe the Pima Indians Diabetes dataset: 768 samples, 8 numerical features (e.g. glucose concentration, BMI, age); binary target: positive (268 positive, 500 negative). Mild class imbalance ($\approx 35\%$ positive). Source: OpenML / Smith et al. [19]. Same split and scaling as Breast Cancer.

5.1.3 Bank Marketing

TODO: Describe the Bank Marketing dataset: Full dataset: ≈ 4521 samples; after 10 %-subsample (fixed per seed): ≈ 452 samples; 16 features after one-hot encoding of categorical variables. Binary target: subscription of a term deposit (yes/no); strong class imbalance ($\approx 88\%$ negative $\rightarrow$ AUROC especially informative). Source: OpenML / Moro et al. [13]. Same split and scaling pipeline as Breast Cancer.

5.2 Experimental Setup

TODO: Describe the full experimental design: 2 models $\times$ 2 optimisers (BO, ACRS) $\times$ 3 seeds $\times$ 3 datasets = 36 runs, 40 trials each (= 1440 total training evaluations).

Tab. 5.1.: Full experimental design: 36 runs, 1440 training evaluations in total.

Model	Optimizer	Seeds	Trials/Run	Runs
MLP	Bayesian (TPE)	42, 123, 456	40	3 per dataset
MLP	ACRS	42, 123, 456	40	3 per dataset
Random Forest	Bayesian (TPE)	42, 123, 456	40	3 per dataset
Random Forest	ACRS	42, 123, 456	40	3 per dataset
Total	–	–	–	36 (1440 evals)

TODO: Insert hardware information: platform, processor, Python version, PyTorch version, CUDA available, GPU. Software stack: Python X.X, PyTorch X.X, scikit-learn X.X, Ray Tune X.X, Optuna X.X, CodeCarbon X.X.

TODO: Describe the reproducibility setup: all relevant random number generators (Python, NumPy, PyTorch, CUDA) are seeded before each run. CuDNN is set to deterministic mode. Seeds: {42, 123, 456}.

TODO: Describe the evaluation protocol: HPO target metric is Val-AUROC; final model uses best hyperparameters per run; test set used only for final evaluation (no influence on selection); aggregation over seeds: median + IQR or mean + std; convergence plot: best-so-far Val-AUROC vs. trial; efficiency plot: cumulative training time vs. AUROC.

Results

TODO: Write introductory paragraph: reference experimental design (Table 5.1), announce structure of this chapter.

6.1 Model Quality

TODO: Write introductory paragraph for this section: refer to Table 6.1 for exact numbers and Figure 6.1 for the seed-wise distribution. Announce that AUROC is the primary metric (optimised during HPO) and Accuracy serves as an interpretability-friendly secondary metric.

Tab. 6.1.: Best Val-AUROC, Val-Accuracy, and convergence speed (trials needed to reach 95 % of the run-maximum AUROC) for all model $\times$ optimizer combinations. Values are mean $\pm$ std over 3 seeds (42, 123, 456). Values are mean $\pm$ std over 3 seeds.

Model	Optimizer	Val-AUROC (mean $\pm$ std)	Val-Accuracy (mean $\pm$ std)	Trials to 95 % (mean $\pm$ std)
<i>Breast Cancer Wisconsin</i>				
MLP	BO	–	–	–
MLP	ACRS	–	–	–
RF	BO	–	–	–
RF	ACRS	–	–	–
<i>Pima Indians Diabetes</i>				
MLP	BO	–	–	–
MLP	ACRS	–	–	–
RF	BO	–	–	–
RF	ACRS	–	–	–
<i>Bank Marketing</i>				
MLP	BO	–	–	–
MLP	ACRS	–	–	–
RF	BO	–	–	–
RF	ACRS	–	–	–

TODO: Fill Table 6.1 with actual values: group by (dataset, model, optimizer), compute mean $\pm$ std over seeds for best AUROC, best Accuracy, and trials to 95 % AUROC. The *Trials to 95 %* column answers: how many trials does each method need before reaching 95 % of its own run-maximum AUROC? A lower value indicates faster convergence.

Combined boxplot: Val-AUROC (top) and Val-Accuracy (bottom) per seed (BO vs. ACRS)

`results/comparison_boxplot_breast_cancer.png`

Fig. 6.1.: Best Val-AUROC (top row) and best Val-Accuracy (bottom row) per seed for the two optimizers (BO, ACRS), split by model (Breast Cancer dataset). Each box summarises 3 seeds; the central line is the median.

TODO: Interpret Figure 6.1: which optimizer achieves higher AUROC and higher Accuracy? Do the two metrics agree on the winner? How large is the difference, and is it consistent across seeds (tight boxes vs. large spread)?

6.2 Cross-Dataset Overview

TODO: Refer to Table 6.1 and Figures 6.2–6.3 for the cross-dataset overview. Discuss: does one optimizer consistently win across all three datasets, or does the winner change? Does the pattern differ between MLP and RF?

6.3 Convergence Behavior

TODO: Describe best-so-far Val-AUROC vs. trial number. Shaded area = ± 1 std over seeds.

TODO: Interpret Figure 6.4: Does BO converge faster? From which trial onward?

TODO: Interpret Figure 6.5: Does the pattern differ from the MLP?

Cross-dataset bar chart: AUROC (top) and Accuracy (bottom) — MLP

`results/cross_dataset_MLP.png`

Fig. 6.2.: Mean best Val-AUROC (top) and Val-Accuracy (bottom) for the MLP across all three datasets. Bars show mean over 3 seeds; error bars show standard deviation. BO vs. ACRS.

6.4 Resource Efficiency

6.4.1 Training Time

TODO: Report cumulative training time per optimizer $\times$ model (mean $\pm$ std over seeds).

TODO: Interpret Figure 6.6: Is the BO overhead from the surrogate model visible?

6.4.2 Energy Consumption

TODO: Report energy consumption (kWh), measured via CodeCarbon [17]. If values = 0.0 for many trials, note measurement precision limitations.

TODO: Comment on values in Figure 6.7 and note measurement limitations.

6.5 Quality–Efficiency Trade-off

TODO: Describe the efficiency plot: cumulative training time (X) vs. best AUROC (Y). Shows which method is better at equal time budget.

Cross-dataset bar chart: AUROC (top) and Accuracy (bottom) — RF

`results/cross_dataset_RF.png`

Fig. 6.3.: Mean best Val-AUROC (top) and Val-Accuracy (bottom) for the Random Forest across all three datasets. Bars show mean over 3 seeds; error bars show standard deviation. BO vs. ACRS.

TODO: Interpret Figure 6.8: From which time point does the BO overhead pay off?

TODO: Interpret Figure 6.9 and compare with MLP.

6.6 Hyperparameter Sampling Distributions

TODO: Describe histograms of sampled hyperparameter values. Key question: Does BO concentrate its samples in different regions than ACRS?

TODO: Comment on Figure 6.10: Does BO concentrate its samples in specific regions?

TODO: Comment on Figure 6.11: Compare sampling patterns with MLP.

Convergence curve MLP: best-so-far Val-AUROC vs. trial

`results/convergence_breast_cancer_MLP.png`

Fig. 6.4.: Convergence of the MLP: best Val-AUROC achieved so far as a function of trial number, averaged over 3 seeds (shaded area = ± 1 standard deviation). Two optimizers (BO, ACRS).

Convergence curve RF: best-so-far Val-AUROC vs. trial

`results/convergence_breast_cancer_RF.png`

Fig. 6.5.: Convergence of the Random Forest: best Val-AUROC achieved so far as a function of trial number, averaged over 3 seeds. Two optimizers (BO, ACRS).

Bar chart: cumulative training time per method $\times$ model

`results/training_time_breast_cancer.png`

Fig. 6.6.: Cumulative training time (40 trials, averaged over 3 seeds) for the two optimizers (BO, ACRS). Error bars show standard deviation across seeds.

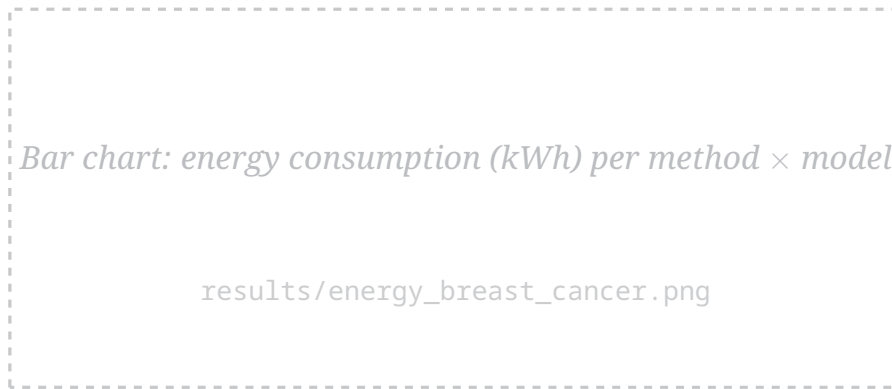


Fig. 6.7.: Total energy consumption (kWh) across all trials (averaged over 3 seeds), measured with CodeCarbon [17]. Two optimizers (BO, ACRS).

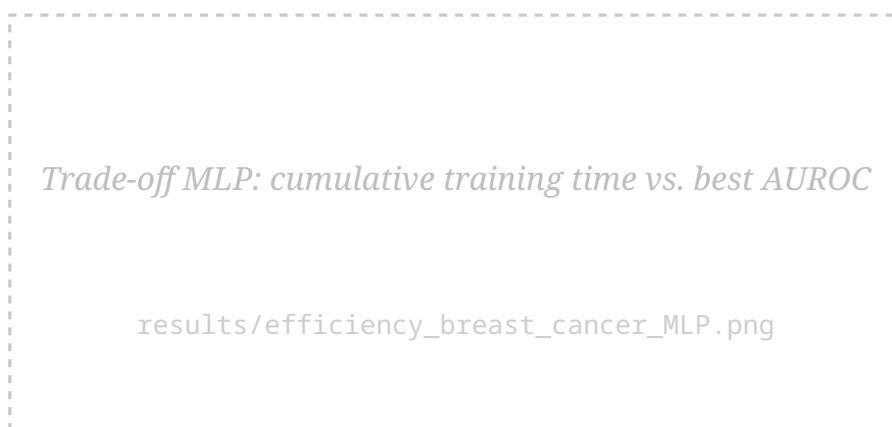


Fig. 6.8.: Trade-off between cumulative training time and best Val-AUROC (MLP). Each line corresponds to one seed; colour encodes the optimizer (BO, ACRS).

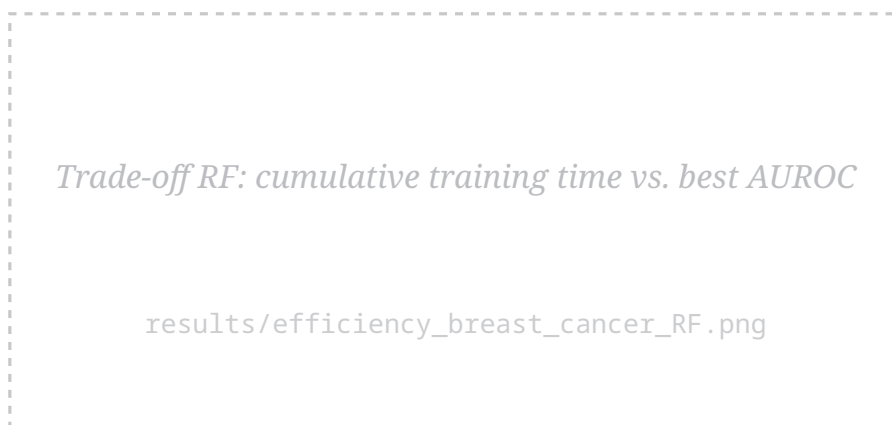


Fig. 6.9.: Trade-off between cumulative training time and best Val-AUROC (Random Forest). Two optimizers (BO, ACRS).

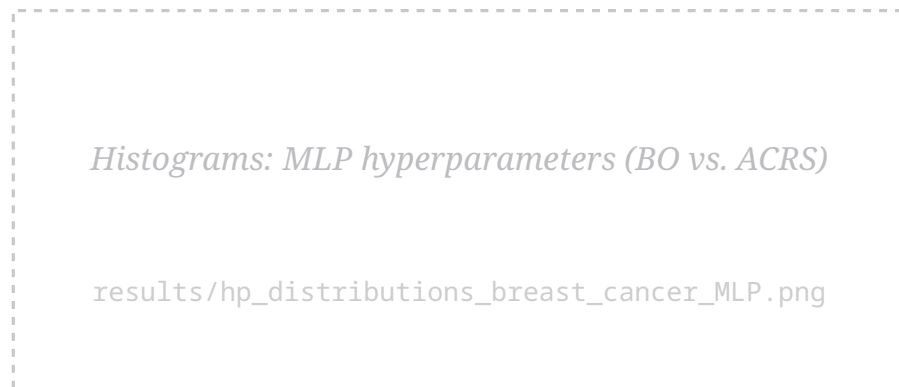


Fig. 6.10.: Distribution of sampled hyperparameters for the MLP. Two optimizers (BO, ACRS).

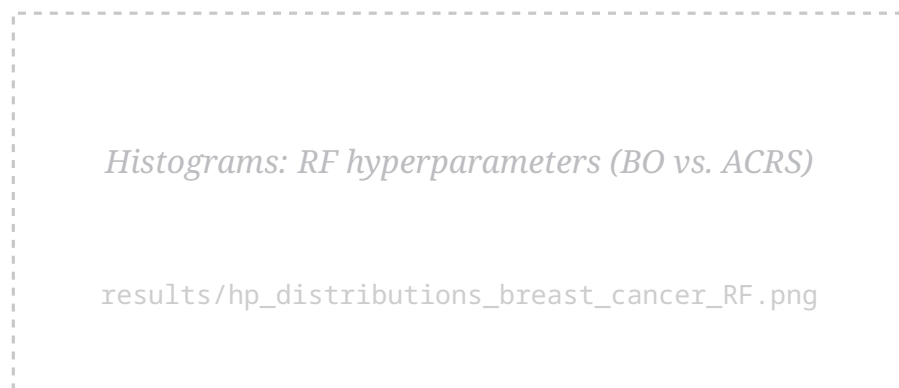


Fig. 6.11.: Distribution of sampled hyperparameters for the Random Forest. Two optimizers (BO, ACRS).

Discussion

TODO: Write introductory paragraph placing the results in the context of the research questions from Section ??.

7.1 Comparison of Optimization Methods

TODO: Discuss model quality (AUROC, Accuracy): Is BO or ACRS statistically significantly better than the other? (Wilcoxon / t -test across seeds.) Does this hold for both MLP and RF across all three datasets? Reference [23] (No Free Lunch) and [2] (TPE). Discuss resource efficiency: Does the quality difference justify the BO overhead from the surrogate model? Does ACRS offer a favourable trade-off between adaptive coordinate-wise sampling and lightweight model-free operation compared to BO? Compare total cost: training time $\times$ trials. Does BO or ACRS converge faster? (Reference Figure 6.4.) Analyse the convergence curves: After how many trials is no further progress measurable for each method? Would 20 trials suffice? $\rightarrow$ Practical implication for budget-constrained settings. Difference MLP vs. RF: RF tends to converge faster (no iterative training). References 6.4, 6.5.

7.2 Result Stability Across Seeds

TODO: Analyse variance across seeds [42, 123, 456]: Which method is more stable (smaller std)? High variance $\rightarrow$ unreliable HPO method for this dataset. Practical consequence: How many seeds are sufficient? Reference boxplot Figure 6.1.

7.3 Limitations

TODO: State the limitations of this study: (1) Datasets: three small tabular datasets (Breast Cancer, Diabetes, Bank Marketing) improve generalisability over single-dataset studies, but results may not transfer to high-dimensional, multi-class, or time-series data; (2) MLP architecture: 1–3 hidden layers cover shallow-to-medium depth, but very deep networks are excluded $\rightarrow$ no conclusions about large-scale deep learning; (3) Trial budget: 40 trials may be insufficient for the 8-dimensional MLP and 7-dimensional RF search spaces, potentially underestimating BO's advantage; (4) Energy measurement: CodeCarbon on CPU-only hardware may be imprecise; (5) No cross-validation (fixed train/val split only); (6) No formal statistical significance testing.

Conclusion

TODO: Write a brief introductory sentence summarising the thesis.

8.1 Summary of Findings

TODO: Summarise the key results in one paragraph: Which method yields better model quality — both in terms of AUROC (primary, discrimination ability) and Accuracy (secondary, interpretability)? Do the two metrics agree on the winner, or do they diverge (e.g. one method wins on AUROC but not on Accuracy)? Which method is more resource-efficient (time, energy)? Do the same conclusions hold for MLP and RF, or are they model-dependent? How stable are the results across seeds? Link back to the research questions from Section ??.

8.2 Practical Implications and Future Work

TODO: Derive practical recommendations: When does BO pay off (structured search space, sufficient trial budget for the surrogate model to learn)? When does ACRS offer a better alternative (limited budget, lightweight model-free operation, no surrogate overhead)? From which trial count does BO's advantage over ACRS become measurable? How do findings differ between MLP (continuous, high-dimensional search space) and Random Forest (mixed categorical/continuous)? Energy consideration: relevant when running many experiments [15].

Outline future directions: (1) More diverse datasets (multi-class, high-dimensional, image, time series) to further probe generalisability beyond small tabular benchmarks; (2) Additional HPO methods: Hyperband [11], SMAC [9], Population-Based Training; (3) Deeper MLP architectures (more than 3 hidden layers, residual connections); (4) Multi-objective optimisation: simultaneously optimising AUROC and energy; (5) Cross-validation instead of fixed val split; (6) Formal statistical significance tests.

Bibliography

- [1]Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631 (cit. on pp. 5, 14).
- [2]James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. “Algorithms for Hyper-Parameter Optimization”. In: *Advances in Neural Information Processing Systems 24 (NeurIPS)*. 2011, pp. 2546–2554 (cit. on pp. 3–5, 14, 29).
- [3]James Bergstra and Yoshua Bengio. “Random Search for Hyper-Parameter Optimization”. In: *Journal of Machine Learning Research* 13 (2012), pp. 281–305 (cit. on pp. 1, 4, 6).
- [4]Leo Breiman. “Bagging Predictors”. In: *Machine Learning* 24.2 (1996), pp. 123–140 (cit. on p. 12).
- [5]Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32 (cit. on p. 12).
- [6]Tom Fawcett. “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874 (cit. on p. 10).
- [7]Matthias Feurer and Frank Hutter. “Hyperparameter Optimization”. In: *Automated Machine Learning: Methods, Systems, Challenges*. Ed. by Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. Springer, 2019, pp. 3–33 (cit. on pp. 1, 3, 4, 6, 9).
- [8]Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016 (cit. on p. 11).
- [9]Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Sequential Model-Based Optimization for General Algorithm Configuration”. In: *Proceedings of the 5th International Conference on Learning and Intelligent Optimization (LION)*. 2011, pp. 507–523 (cit. on pp. 3, 5, 31).
- [10]Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)* (2015) (cit. on p. 12).
- [11]Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization”. In: *Journal of Machine Learning Research* 18 (2018), pp. 1–52 (cit. on pp. 3, 6, 31).

- [12]Richard Liaw, Eric Liang, Robert Nishihara, et al. “Tune: A Research Platform for Distributed Model Selection and Training”. In: *arXiv preprint arXiv:1807.05118* (2018) (cit. on p. 5).
- [13]Sérgio Moro, Paulo Cortez, and Paulo Rita. “A Data-Driven Approach to Predict the Success of Bank Telemarketing”. In: *Decision Support Systems* 62.1 (2014), pp. 22–31 (cit. on p. 19).
- [14]Vu Nguyen. “Bayesian Optimization for Accelerating Hyper-Parameter Tuning”. In: *Proceedings of the 2nd IEEE International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*. 2019, pp. 302–305 (cit. on p. 5).
- [15]David Patterson, Joseph Gonzalez, Quoc Le, et al. “Carbon Emissions and Large Neural Network Training”. In: *arXiv preprint arXiv:2104.10350* (2021) (cit. on p. 31).
- [16]Lutz Prechelt. “Early Stopping—But When?” In: *Neural Networks: Tricks of the Trade*. Ed. by Genevieve Orr and Klaus-Robert Müller. Springer, 1998, pp. 55–69 (cit. on p. 12).
- [17]Victor Schmidt, Kamal Goyal-Kamal, Joachim Goyal, et al. “CodeCarbon: Estimate and Track Carbon Emissions from Machine Learning Computing”. In: *arXiv preprint arXiv:2002.05651* (2022) (cit. on pp. 7, 11, 23, 26).
- [18]Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. “Taking the Human Out of the Loop: A Review of Bayesian Optimization”. In: *Proceedings of the IEEE* 104.1 (2016), pp. 148–175 (cit. on pp. 3, 5).
- [19]J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes. “Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus”. In: *Proceedings of the Annual Symposium on Computer Application in Medical Care*. 1988, pp. 261–265 (cit. on p. 19).
- [20]Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. “Practical Bayesian Optimization of Machine Learning Algorithms”. In: *Advances in Neural Information Processing Systems 25 (NeurIPS)*. 2012, pp. 2951–2959 (cit. on pp. 3, 5, 15).
- [21]Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15 (2014), pp. 1929–1958 (cit. on p. 11).
- [22]W. H. Wolberg, W. N. Street, and O. L. Mangasarian. *Breast Cancer Wisconsin (Diagnostic) Data Set*. UCI Machine Learning Repository. 1995 (cit. on p. 19).
- [23]David H. Wolpert and William G. Macready. “No Free Lunch Theorems for Optimization”. In: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), pp. 67–82 (cit. on pp. 1, 6, 10, 29).
- [24]Stephen J. Wright. “Coordinate Descent Algorithms”. In: *Journal of Optimization Theory and Applications* 151.3 (2015), pp. 723–746 (cit. on p. 5).

- [25]Li Yang and Abdallah Shami. “On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice”. In: *Neurocomputing* 415 (2020), pp. 295–316 (cit. on pp. 3, 4, 6).

List of Figures

6.1.	Best Val-AUROC (top row) and best Val-Accuracy (bottom row) per seed for the two optimizers (BO, ACRS), split by model (Breast Cancer dataset). Each box summarises 3 seeds; the central line is the median.	22
6.2.	Mean best Val-AUROC (top) and Val-Accuracy (bottom) for the MLP across all three datasets. Bars show mean over 3 seeds; error bars show standard deviation. BO vs. ACRS.	23
6.3.	Mean best Val-AUROC (top) and Val-Accuracy (bottom) for the Random Forest across all three datasets. Bars show mean over 3 seeds; error bars show standard deviation. BO vs. ACRS.	24
6.4.	Convergence of the MLP: best Val-AUROC achieved so far as a function of trial number, averaged over 3 seeds (shaded area = ± 1 standard deviation). Two optimizers (BO, ACRS).	25
6.5.	Convergence of the Random Forest: best Val-AUROC achieved so far as a function of trial number, averaged over 3 seeds. Two optimizers (BO, ACRS).	25
6.6.	Cumulative training time (40 trials, averaged over 3 seeds) for the two optimizers (BO, ACRS). Error bars show standard deviation across seeds.	25
6.7.	Total energy consumption (kWh) across all trials (averaged over 3 seeds), measured with CodeCarbon [17]. Two optimizers (BO, ACRS).	26
6.8.	Trade-off between cumulative training time and best Val-AUROC (MLP). Each line corresponds to one seed; colour encodes the optimizer (BO, ACRS).	26
6.9.	Trade-off between cumulative training time and best Val-AUROC (Random Forest). Two optimizers (BO, ACRS).	26
6.10.	Distribution of sampled hyperparameters for the MLP. Two optimizers (BO, ACRS).	27
6.11.	Distribution of sampled hyperparameters for the Random Forest. Two optimizers (BO, ACRS).	27

List of Tables

- 4.1. Hyperparameter search space of the MLP. 13
- 4.2. Hyperparameter search space of the Random Forest. 14
- 5.1. Full experimental design: 36 runs, 1440 training evaluations in total. 20
- 6.1. Best Val-AUROC, Val-Accuracy, and convergence speed (trials needed to reach 95 % of the run-maximum AUROC) for all model $\times$ optimizer combinations. Values are mean $\pm$ std over 3 seeds (42, 123, 456). Values are mean $\pm$ std over 3 seeds. 21

List of Listings

Appendix

A

A.1 Complete Results Tables

TODO: Insert table of all trial results (or top-10 per run). Columns: model, optimizer, seed, trial number, hyperparameters, Val-AUROC, Val-Accuracy, training time, inference time, energy (kWh).

A.2 Best Hyperparameters per Configuration

TODO: Insert table: model $\times$ optimizer $\times$ seed $\rightarrow$ best found HP combination + achieved AUROC on test set.

A.3 Hardware and Software Environment

TODO: Insert hardware information: platform, processor, Python version, PyTorch version, CUDA available, GPU.

A.4 Reproducibility Checklist

TODO: Short checklist: All random number generators seeded before each run: Yes. CuDNN deterministic mode enabled: Yes. Dependency versions pinned: TODO (currently unpinned). Random-sample order: controlled by the respective optimization framework.

Colophon

This thesis was typeset with $\text{\LaTeX} 2_{\epsilon}$. It uses the *Clean Thesis* style developed by Ricardo Langner. The design of the *Clean Thesis* style is inspired by user guide documents from Apple Inc.

Download the *Clean Thesis* style at <http://cleanthesis.der-ric.de/>.

Adaptations to the style of the Institute of Computer Science can be found at <https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu>.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen¹ stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden². Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, March 24, 2026

Dennis Diehl

¹Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

²Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Use of AI tools.

AI Tool	Used for	Reason	When

